

Quorum — AI Product Council

Build spec (weekend v1)

Positioning: Existing “AI councils” aggregate opinions into an answer. Quorum governs a decision — every feature request gets a task contract, a policy check, adversarial review, and an auditable change receipt before it’s considered “decided.” Harness engineering applied to product management, not just code.

Fictional company: Fenwick Digital (generic B2B SaaS) — keeps this distinct from Northbridge/Meridian and lets the demo read as a standalone platform, not another insurance vertical.

1. Data model (Postgres)

agents

id, name, role, status (green|yellow|gray|orange), runtime (claude), created_at

tasks

id, title, objective, scope (text[]), constraints (text[]),
acceptance_criteria (text[]), policy_tier (auto|approval_required|prohibited),
state (proposed|contracted|in_progress|in_review|approved|rejected|blocked),
owner_agent_id, thread_id, created_at, updated_at

threads

id, task_id

messages

id, thread_id, author_type (agent|human), author_id,
message_type (contract|proposal|critique|decision|receipt),
content, created_at

receipts

id, task_id, objective, changed (text[]), verified (text[]),
not_verified (text[]), risks (text[]), approval_needed (text[]),
decided_by, decided_at

policy_rules

id, action_type, tier, rationale

2. Task state machine

```
proposed
  → PM agent drafts contract →
contracted
  → Engineering + Risk agents claim in parallel →
in_progress
  → both post outputs to thread →
in_review
  → Reviewer agent checks →
    approved → receipt generated, task closed
    rejected → sent back to specific author w/ reason → in_progress
blocked (any state) — escalation condition met, needs human
```

3. Agent briefs (4 agents, Claude Sonnet, role-scoped system prompts)

PM Agent — owns intake. Converts a raw feature request into a task contract (objective, scope, constraints, acceptance criteria). Does not estimate effort or assess risk. Hands off to Engineering + Risk once contract exists.

Engineering Lead Agent — owns technical feasibility. Reads the contract, proposes an approach, flags dependencies and effort. Cannot approve its own estimate.

Risk/Governance Agent — owns the policy tier. Checks the contract against policy_rules (customer-facing? pricing? data handling? irreversible?) and assigns auto / approval_required / prohibited. This tier gates what the Reviewer can approve without escalating to a human.

Reviewer Agent — independent, never authors. Rejects on: missing acceptance criteria, unsourced technical claims, scope creep past the contract, policy tier violated without escalation. Approves only when all pass — then generates the receipt.

4. Policy tiers (example rubric)

Tier	Examples
Auto	Research, internal docs, competitive analysis
Approval required	Roadmap date commitments, customer-facing copy, pricing

Prohibited (hard gate)	Agent cannot approve its own work; cannot publish externally; cannot commit legal/compliance language
---------------------------	--

5. Stack (matches your standard build pattern)

- Postgres (schema above)
- FastAPI on Railway — task/thread/message CRUD, state machine, Claude API calls per agent role
- Lovable frontend — Slack-style board: Channels = product areas, Tasks = council items with status dot, Threads = deliberation, Receipt card on approval
- Claude Sonnet via Anthropic API, one system prompt per role above

6. Weekend build order

1. Sat AM: schema + FastAPI CRUD + state machine
2. Sat PM: 4 agent system prompts + Claude API wiring, seed 3-4 sample feature requests
3. Sun AM: Lovable board UI (task list, thread view, status dots, receipt card)
4. Sun PM: run end-to-end, fix rejection loop, write up LinkedIn post + README framing the harness/governance angle